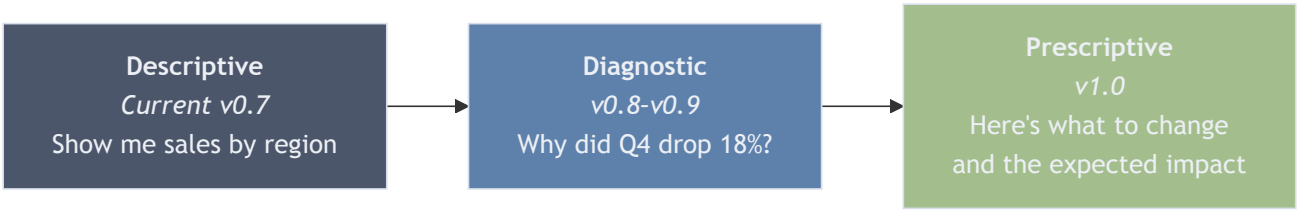
Spectra Product Expansion — Business Optimization Capabilities

The Problem

Spectra today is a **reactive analysis tool** — users upload data, ask questions, get answers. To become a true **business optimization platform** and differentiate from tools like Julius.ai, Spectra needs to move up the analytics maturity curve:



Key differentiator: Spectra becomes an analyst that works for you — it proactively scans, flags issues, explains causes, and recommends actions. The user's job shifts from "figure out what to ask" to "review and decide."

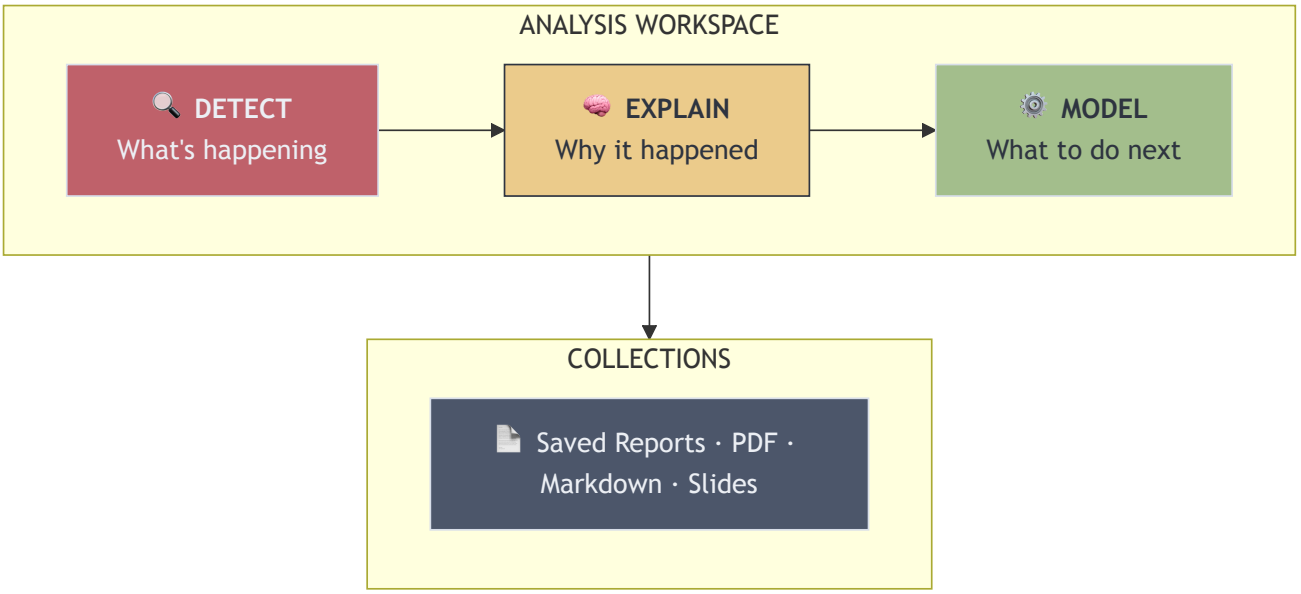
Product Architecture: Three Modules, One Platform

Module 1: Chat Sessions (existing — the base tool)

The current chat-with-your-data flow. Stays as-is. Becomes the most primitive feature of Spectra — freeform exploration, quick questions, ad-hoc analysis. Think of it as the "calculator" — always available, always useful, but not the main event.

Module 2: Analysis Workspace (new — the differentiator)

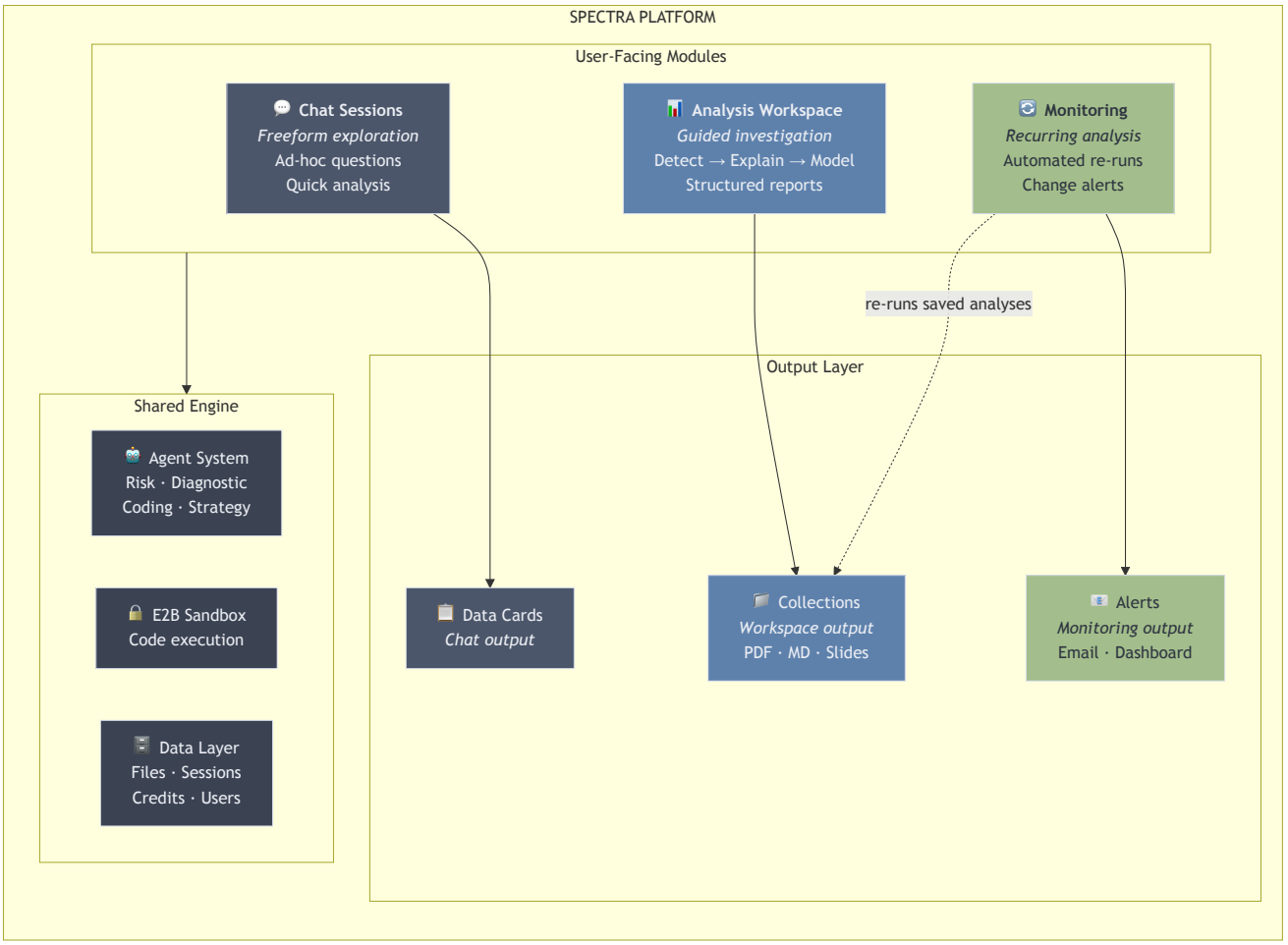
A completely separate module with its own entry point, its own flow, and its own output format. This is where Spectra becomes a business tool, not just a data tool.



Module 3: Monitoring (future — the retention engine)

A separate module for recurring, automated analysis. When data is regularly updated, Spectra re-runs saved analyses and delivers comparison reports — turning one-time analysis into ongoing business intelligence. See Monitoring Module deep-dive below.

Platform Architecture



All three modules share the same data layer, same agents, same E2B engine — but have completely different UX paradigms:

	Chat Sessions	Analysis Workspace	Monitoring
Purpose	Exploration	Deliverables	Ongoing vigilance
Interaction	Freeform typing	Guided steps + Q&A	Set-and-forget
Trigger	User asks a question	User starts an analysis	New data arrives
Output	Data Cards in conversation	Structured reports	Change alerts + comparison reports
Saved as	Chat history	Collections (PDF/MD/slides)	Monitoring dashboard + email alerts
User mindset	"Let me check something"	"I need to produce a report"	"Watch this for me"

Business Executive Assessment

"Would I actually use this, or is it just a demo?"

Perspective: department head, has data in Excel, reports to a VP.

Risk Radar (Detect) — SOLVES A REAL PROBLEM

The most dangerous issues are the ones I *didn't think to check*. Today I open Excel, stare at numbers, and hope I notice something wrong. If Spectra immediately says "3 things you should know about" after I upload my monthly data — that's genuinely valuable. It's the difference between catching a problem in week 1 vs. week 8. **This is not a nice-to-have.**

Insight Engine (Explain) — YES, BUT ONLY IF GUIDED

When my boss asks "why did margins drop?", I spend hours slicing data in Excel. If Spectra walks me through it — asking clarifying questions and progressively narrowing down — that saves real time. **A raw diagnostic dump would be useless. A guided conversation that arrives at an answer is gold.** The Q&A flow is exactly right.

Optimization Studio (Model) — CAUTIOUS YES

I'd use scenario modeling ("what if we raise prices 5%?") weekly. But I'd be skeptical of model recommendations unless I understand how it got there. **Start with simulation (let me play with inputs and see outputs), not prescription (Spectra tells me what to do).** Trust needs to be earned over time.

Collections (saved reports) — THE SLEEPER FEATURE

Half my job is producing reports for stakeholders. Today I copy-paste from tools into PowerPoint. That workflow is broken. If every analysis becomes a polished PDF or slide deck — **that's not a feature, that's the reason I'd pay monthly.** This is what makes Spectra sticky.

Monitoring (recurring analysis) — YES, BUT NOT HOW YOU THINK

I don't have automated data pipelines. I export from our ERP manually each month. What I actually want: upload this month's data, and Spectra automatically compares it to last month's analysis and tells me what changed. **The "automation" I need is one-click re-analysis, not a data pipeline.**

Bottom line: The guided flow (not chat) is the right call. Chat is for exploration. But when I need to produce a deliverable — a risk assessment, a root cause report, an optimization plan — I need a structured process with a structured output.

Apple PM Assessment

"What's the simplest path from data to decision?"

Design philosophy: hide complexity, reveal it progressively, make the default path feel inevitable.

Core UX Principles

1. Progressive disclosure, not feature overload

- User starts at Detect. They *can* go to Explain. They *can* go to Model.
- But if all they need is "flag the risks" — they stop at step 1 and save.
- No one is forced through all three phases.
- Think: Apple Health shows "cardio fitness declining" → tap → contributing factors → tap → recommendations. Each layer is optional.

2. The Q&A flow IS the product

- Not a chatbot conversation. Structured, guided, with discrete choices.
- Each step has ONE clear question and 2-4 clear options.
- Example:

> *Spectra: "Revenue declined 18% in Q4. Which dimension matters most to you?" [Region] [Product] [Channel] [Customer Segment] User picks "Region" Spectra: "The decline is concentrated in APAC (-34%). Want to dig into APAC specifically?"*

- 3-5 exchanges max to reach a root cause. Not an open-ended conversation.

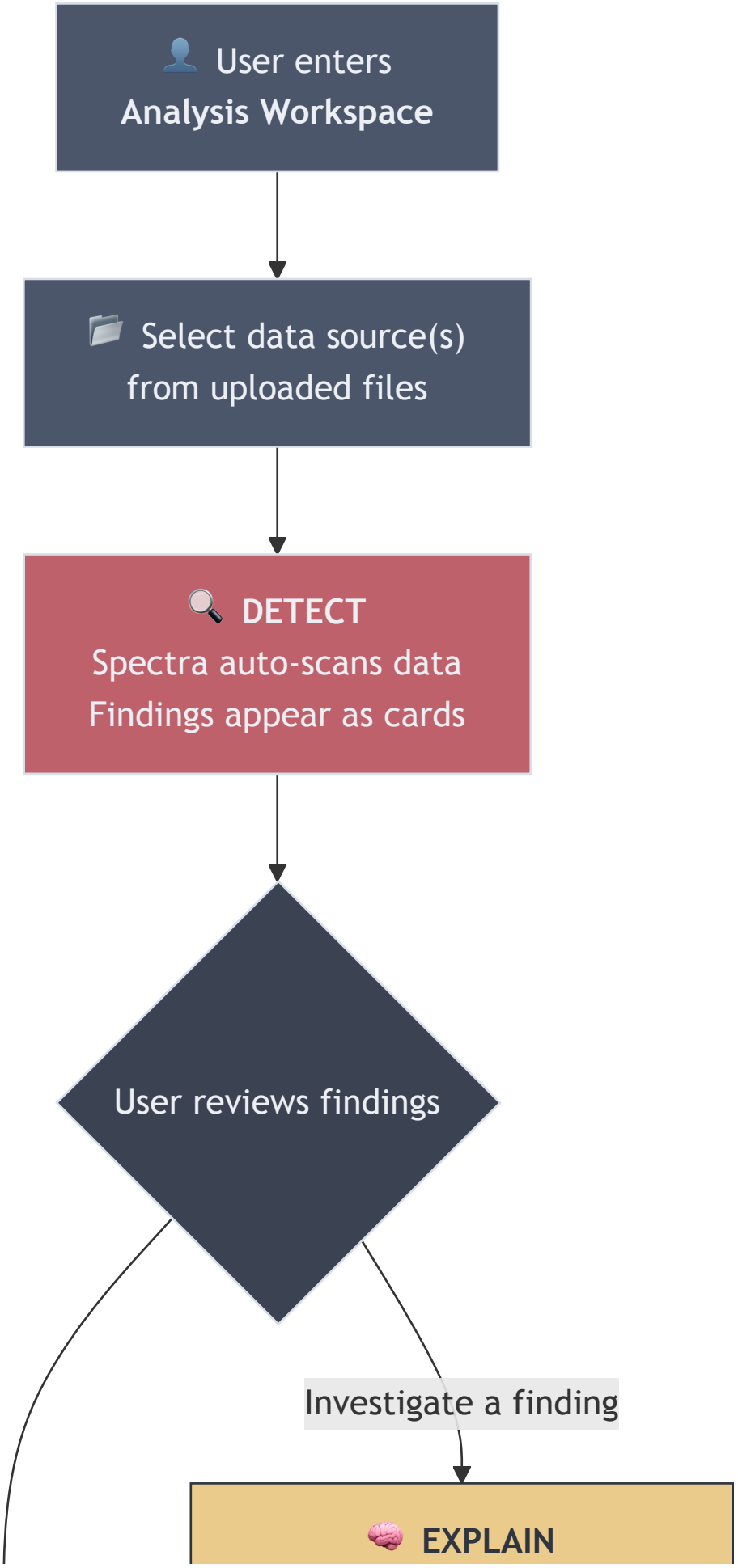
3. Collections is the output, not the analysis

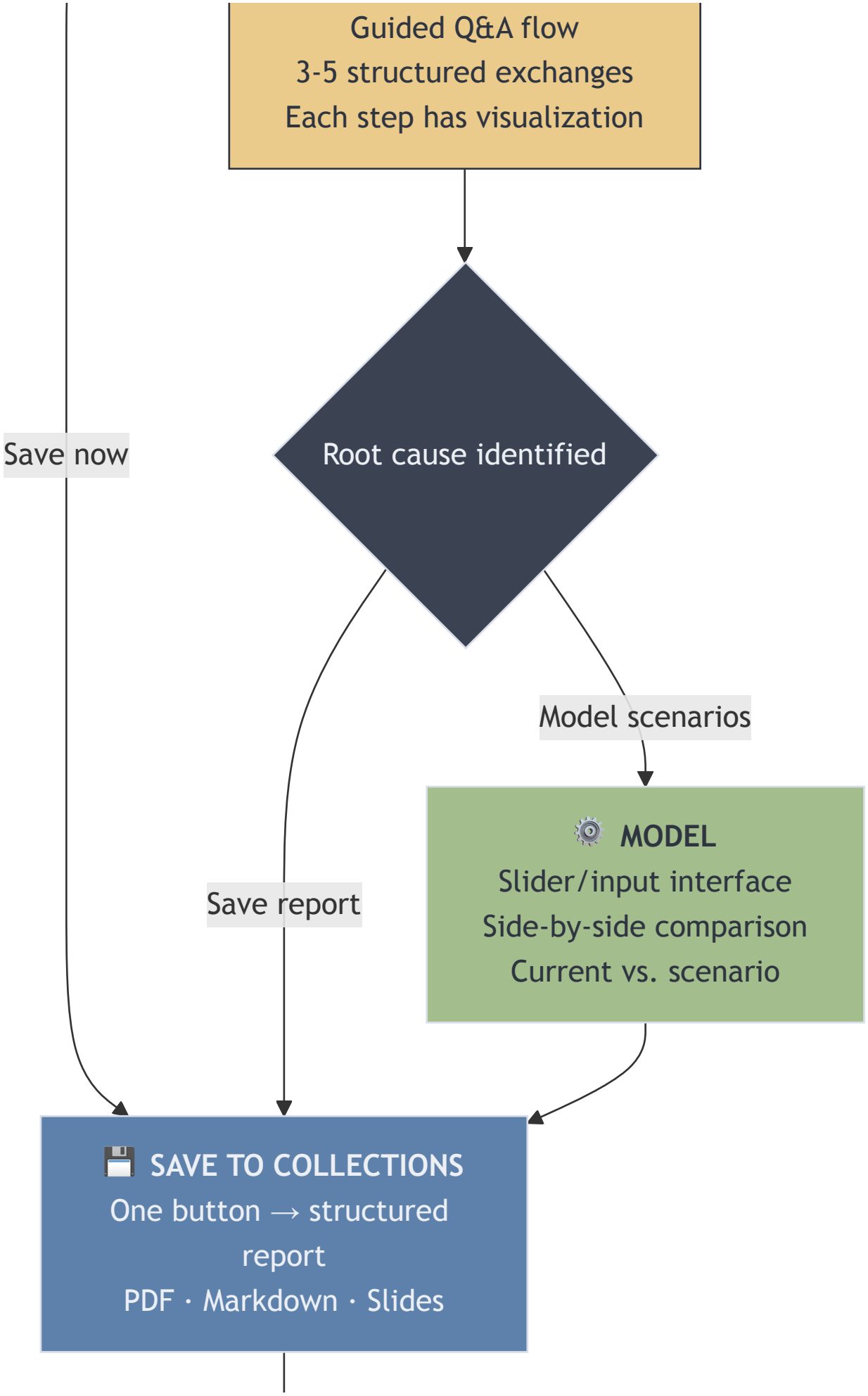
- Users don't come to Spectra for charts. They come for decisions and reports.
- Collections makes Spectra the system of record for data-driven decisions.
- This is what justifies recurring payment — your analysis history lives here.

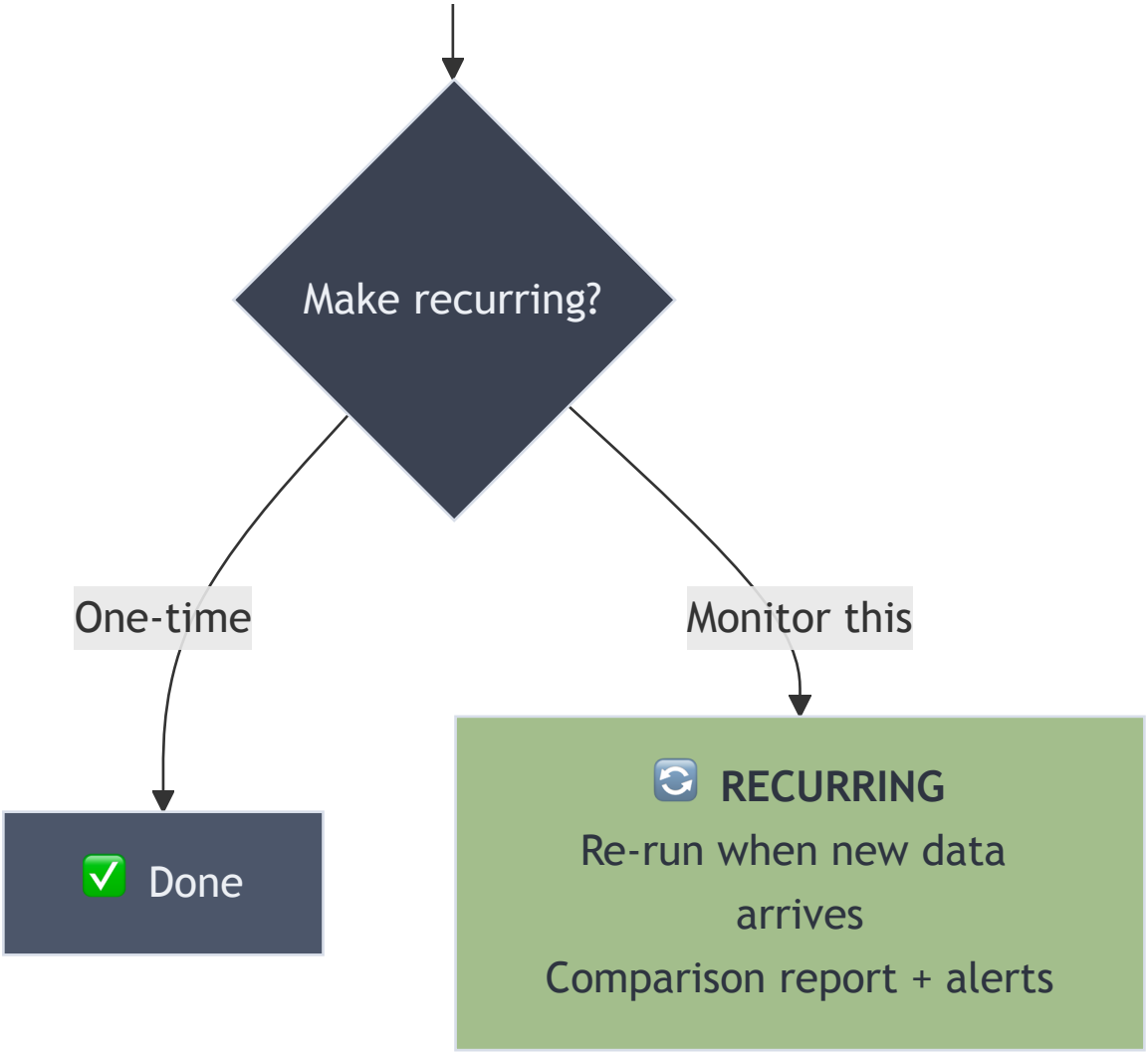
4. Simulation over prescription

- Don't tell users what to do. Give them levers to pull and show them projected outcomes.
- Simple interface: sliders/inputs for key variables.
- Side-by-side: "Current trajectory" vs "Your scenario."
- No ML jargon. No "model training." User adjusts business levers, sees projected results.

The User Journey (end to end)


User enters
Analysis Workspace
Select data source(s)
from uploaded files
DETECT
Spectra auto-scans data
Findings appear as cards
User reviews findings
Investigate a finding
EXPLAIN


Guided Q&A flow
3-5 structured exchanges
Each step has visualization
Root cause identified
Save now
Save report
Model scenarios
MODEL
Slider/input interface
Side-by-side comparison
Current vs. scenario
SAVE TO COLLECTIONS
One button → structured report
PDF · Markdown · Slides



Step 1: Start an Analysis

- User enters Analysis Workspace
- Picks data source(s) from their uploaded files
- Spectra auto-runs detection (Risk Radar)
- Screen shows findings as cards: "Here's what we found"
- No configuration, no setup. Select data → see findings.

Step 2: Guided Investigation (the Q&A flow)

- User taps a finding: "Revenue declined 18% in Q4"
- Spectra asks structured questions with discrete choices (not freeform chat)
- 3-5 exchanges, progressively narrowing to root cause
- Each step includes a visualization supporting the finding
- The conversation IS the analysis

Step 3: Model & Simulate (only when user is ready)

- After diagnosis, Spectra offers: "Want to model a recovery scenario?"
- Slider/input interface for key variables
- Side-by-side comparison: current trajectory vs user's scenario
- No jargon. Business levers in, projected outcomes out.

Step 4: Save to Collections

- One button: "Save as Report"

- Spectra compiles the entire journey — findings, investigation steps, charts, scenario results — into a structured document
- Export as PDF, Markdown, or presentation slides
- Lives in Collections, organized by date/topic

Step 5: Make Recurring (optional)

- After saving, user can mark this analysis as "recurring"
- Next time matching data arrives, Spectra re-runs and diffs automatically
- See Monitoring Module for details

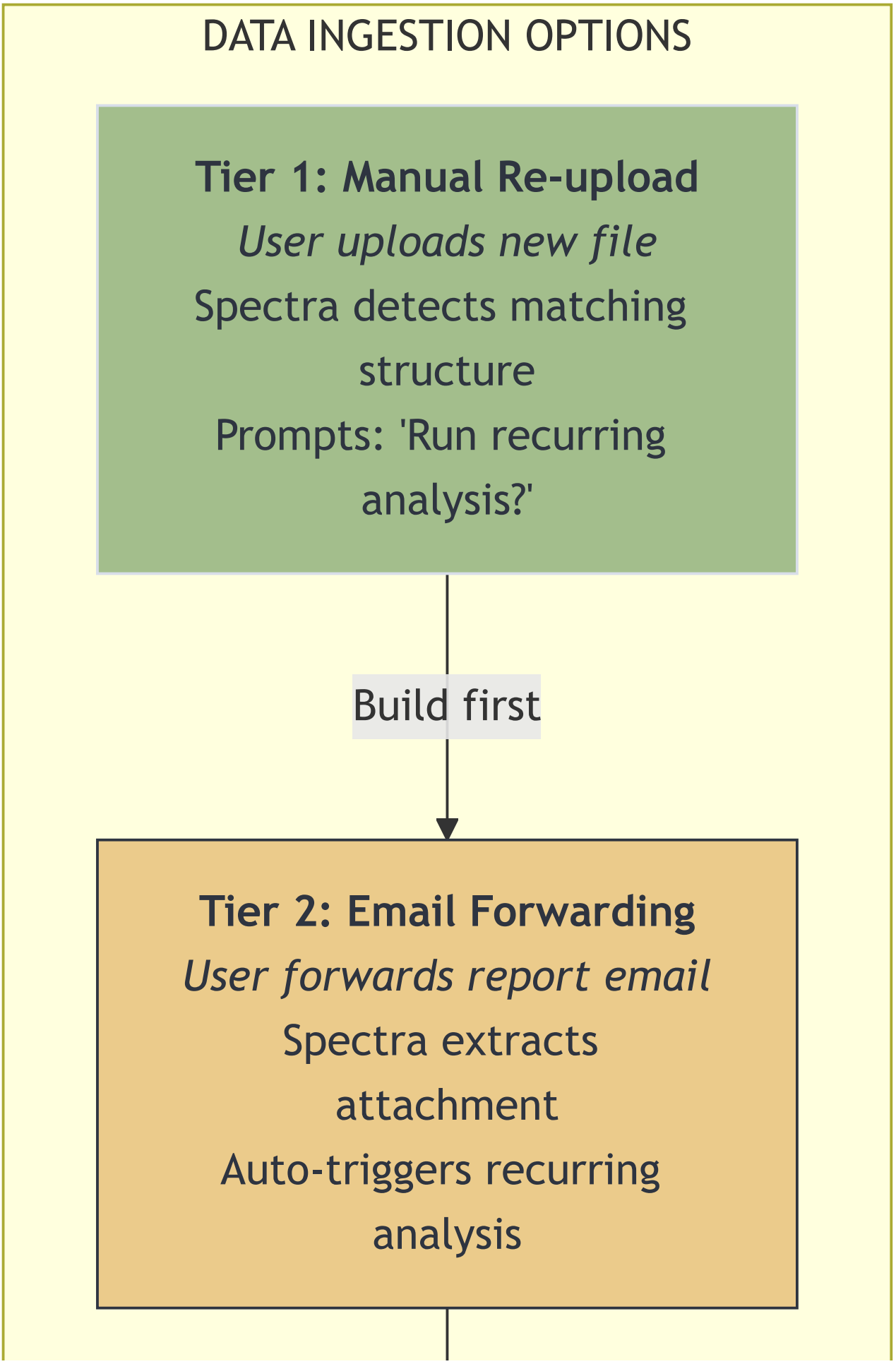
Module 3: Monitoring Deep-Dive

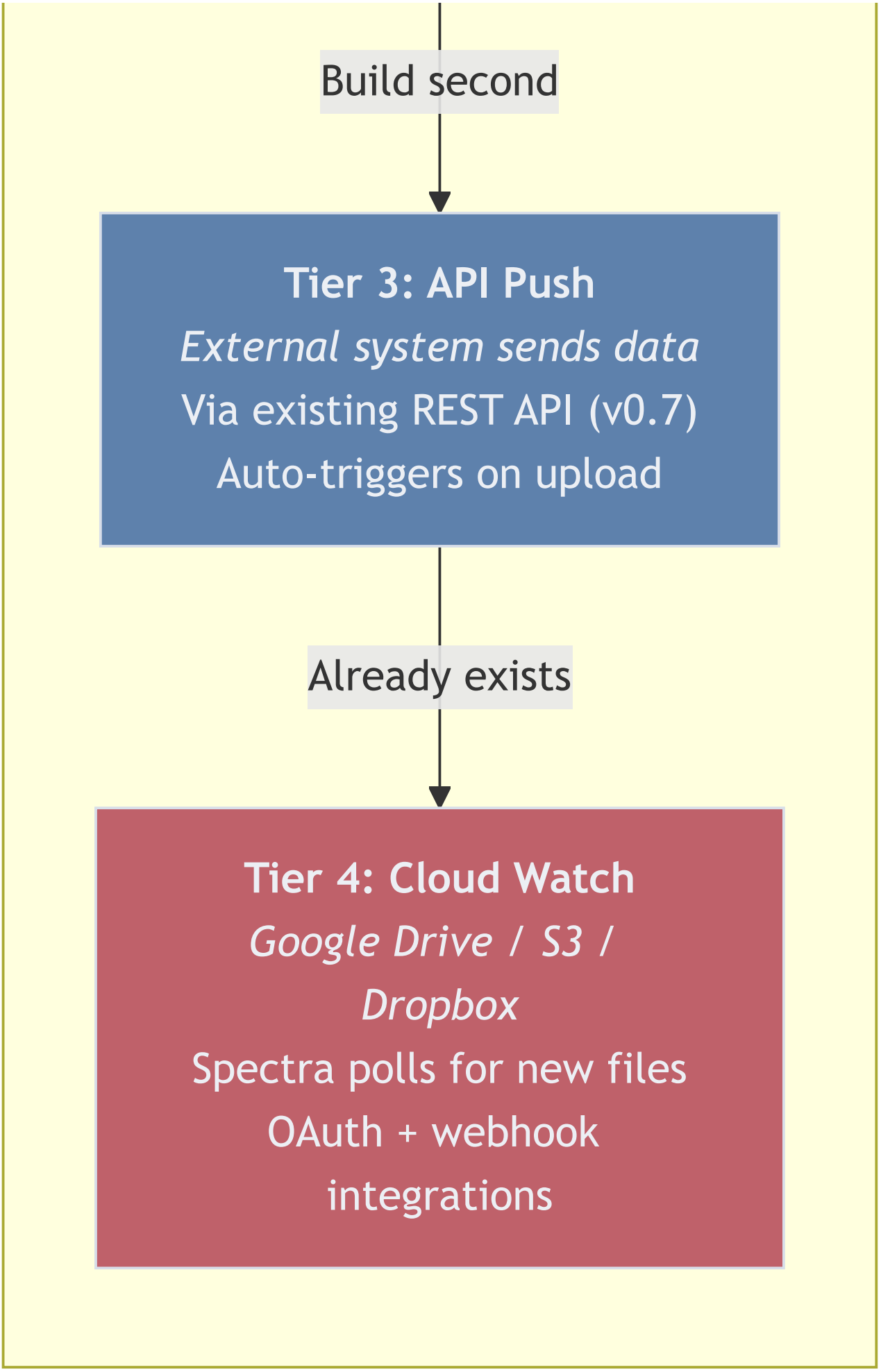
The Concept

When a user completes an analysis and saves it to Collections, they can mark it as "recurring." This tells Spectra: "Run this same analysis again whenever I give you new data with the same structure." The output is a comparison report (what changed since last time) with flagged anomalies, delivered via email and saved to Collections.

The Data Ingestion Challenge

The hardest question isn't "how to re-run analysis" — it's **"how does new data get into Spectra?"**


DATA INGESTION OPTIONS
Tier 1: Manual Re-upload
User uploads new file
Spectra detects matching structure
Prompts: 'Run recurring analysis?'
Build first
Tier 2: Email Forwarding
User forwards report email
Spectra extracts attachment
Auto-triggers recurring analysis


Build second
Tier 3: API Push
External system sends data
Via existing REST API (v0.7)
Auto-triggers on upload
Already exists
Tier 4: Cloud Watch
Google Drive / S3 / Dropbox
Spectra polls for new files
OAuth + webhook integrations

Tier	Method	User Effort	Engineering Effort	Target User
1	Manual re-upload	Low (upload + one click)	Low (structure matching + diff)	Individual analyst
2	Email forwarding	Very low (forward email)	Medium (email ingestion pipeline)	Analyst who gets reports via email
3	API push	None (automated)	Low (already built in v0.7)	Teams with technical capability
4	Cloud storage watch	None (automated)	High (OAuth, polling, webhooks)	Enterprise teams

Recommended approach: Build Tier 1 first, Tier 3 is free (already exists), Tier 2 and 4 are future expansions based on demand.

Why "Recurring Analysis" Before "Automated Pipeline"

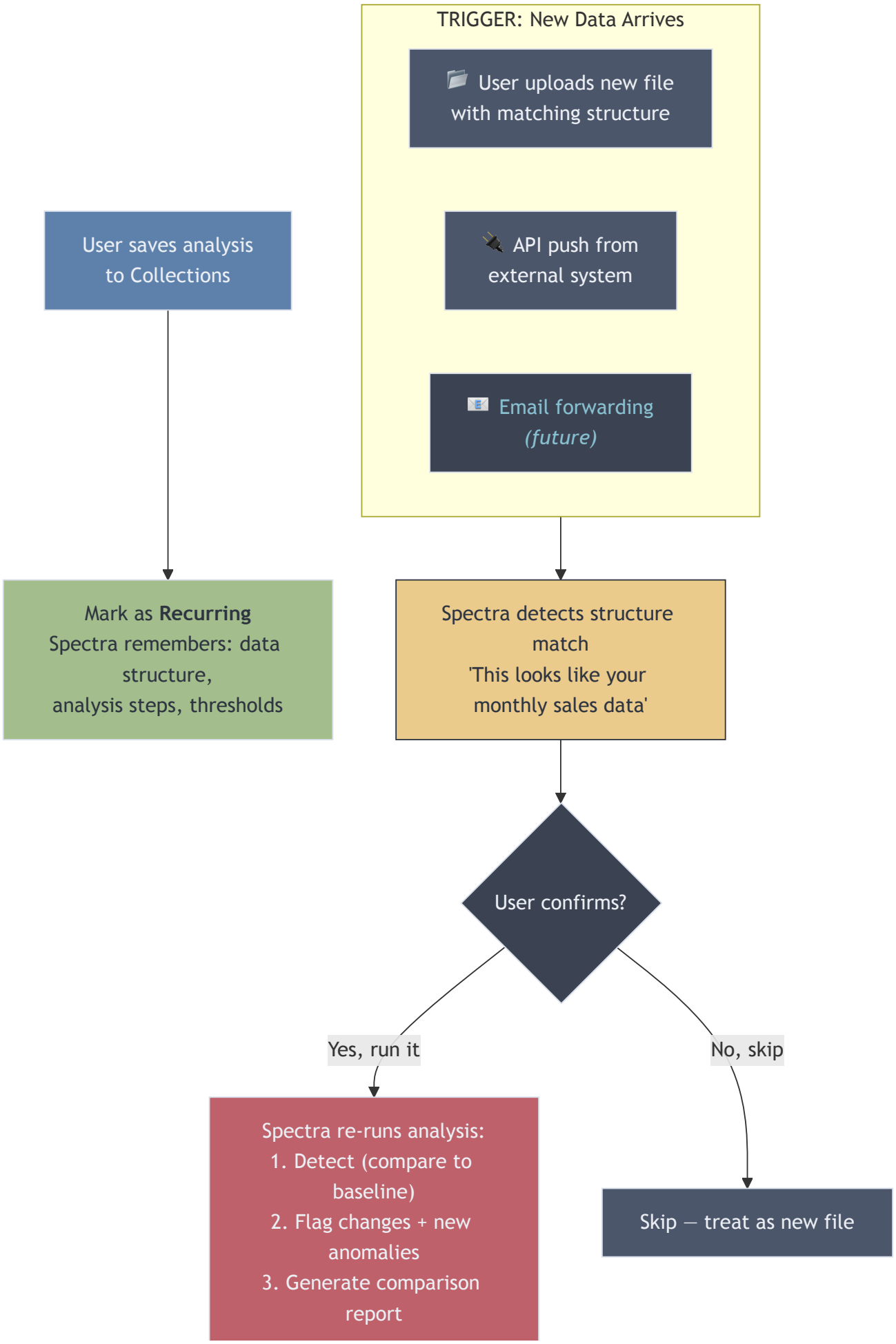
Most target users (business analysts, department heads) don't have automated data pipelines today. Their reality:

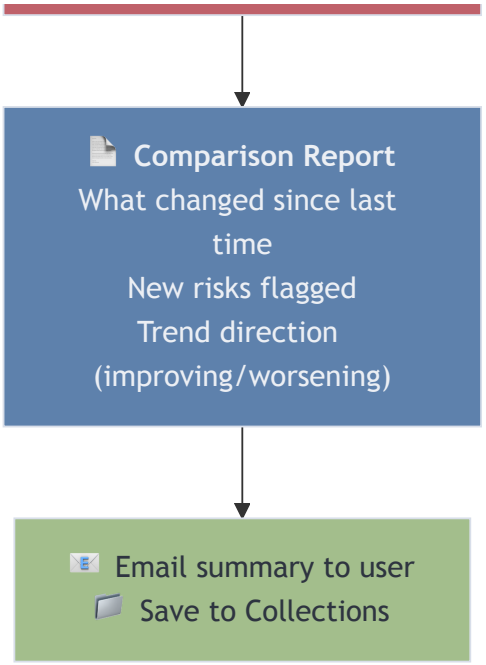
- Export from ERP/CRM manually each month
- Receive a CSV via email from finance
- Download from a dashboard or report tool

For these users, **manual re-upload + intelligent "compare to last time" is the right v1.** It's 80% of the value with 20% of the engineering.

The automated pipeline (email ingestion, cloud storage watch) should come when we have enterprise customers specifically requesting it — and are willing to pay for it.

Monitoring UX Flow


TRIGGER: New Data Arrives
User uploads new file with matching structure
API push from external system
Email forwarding (future)
User saves analysis to Collections
Mark as Recurring
Spectra remembers: data structure,
analysis steps, thresholds
Spectra detects structure match
'This looks like your monthly sales data'
User confirms?
Yes, run it
No, skip
Spectra re-runs analysis:
1. Detect (compare to baseline)
2. Flag changes + new anomalies
3. Generate comparison report
Skip — treat as new file



What the Comparison Report Contains

- **Period-over-period diff:** "Revenue: $1.2M → $1.05M (-12.5%)"
- **New anomalies:** Risks that weren't present last time
- **Resolved anomalies:** Previous risks that have improved
- **Trend direction:** Is each metric improving or worsening vs. previous periods?
- **Flagged thresholds:** Any user-defined thresholds breached (e.g., "alert if churn > 5%")

Credit Model for Monitoring

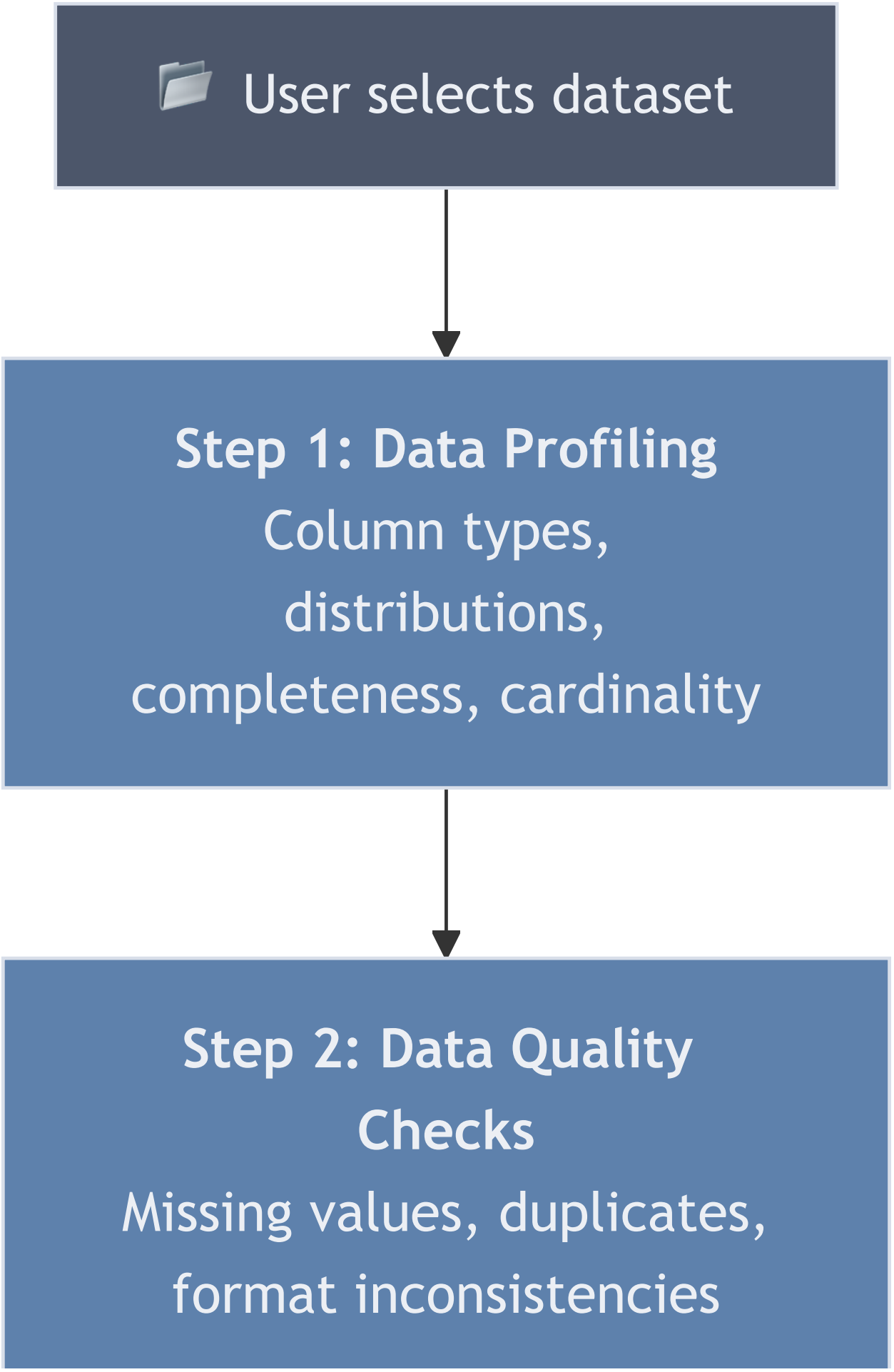
Each recurring analysis re-run costs credits (same as a manual analysis). Users should understand upfront: "This recurring analysis will use ~X credits each time it runs." This prevents surprise credit depletion and aligns cost with value.

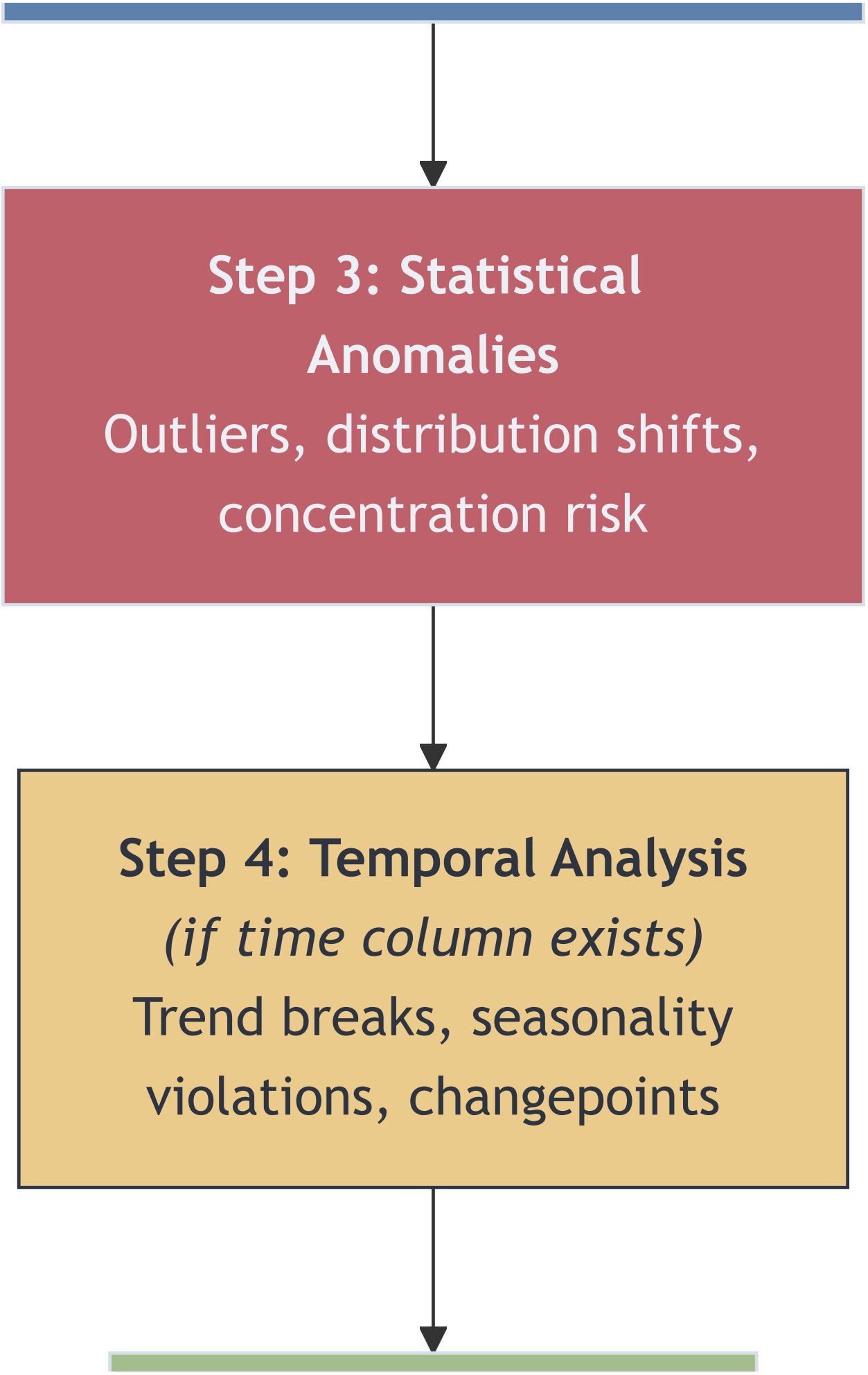
Statistical Methods by Stage

Each stage of the Analysis Workspace uses different statistical techniques. The methods are ordered from simplest (always run) to advanced (run when data supports it). All execution happens in the existing E2B sandbox using Python (pandas, scipy, scikit-learn, statsmodels).

Stage 1: DETECT — Anomaly & Risk Identification

The goal is to answer: **"What should you pay attention to?"** without the user asking. This runs automatically when data is selected.


User selects dataset
Step 1: Data Profiling
Column types, distributions, completeness, cardinality
Step 2: Data Quality Checks
Missing values, duplicates, format inconsistencies


Step 3: Statistical Anomalies
Outliers, distribution shifts, concentration risk
Step 4: Temporal Analysis
(if time column exists)
Trend breaks, seasonality violations, changepoints

🔍 Findings Cards Ranked by severity

Method	What It Catches	When To Use	Python Library
Descriptive profiling	Column types, null rates, unique counts, basic stats (mean, median, std)	Always — first pass on every dataset	`pandas.describe()`, `pandas.dtypes`
Missing value pattern analysis	Systematic gaps (e.g., entire column null after a date, correlated missingness)	Always	`pandas.isnull()`, `missingno`
Duplicate detection	Exact and near-duplicate rows	Always	`pandas.duplicated()`
Z-score outlier detection	Individual values that deviate >2-3 standard deviations from the column mean	Numeric columns with roughly normal distribution	`scipy.stats.zscore`
IQR (Interquartile Range)	Robust outlier detection that works on skewed distributions (values below Q1-1.5*IQR or above Q3+1.5*IQR)	Numeric columns — more robust than Z-score for non-normal data	`pandas` quartile math
Isolation Forest	Multi-dimensional outliers that look normal on individual columns but are unusual in combination	When dataset has 3+ numeric columns	`sklearn.ensemble.IsolationForest`
Herfindahl-Hirschman Index (HHI)	Concentration risk — e.g., "80% of revenue comes from 2 clients"	Categorical columns with associated numeric values	Manual calculation on `pandas.groupby`
Distribution shape analysis	Skewness, kurtosis, bimodality — flags when a column's distribution is unusual or has shifted	Numeric columns with >50 rows	`scipy.stats.skew`, `scipy.stats.kurtosis`
Changepoint detection (PELT)	Abrupt shifts in a time series — e.g., "revenue mean shifted down starting October"	Time-series data with >30 data points	`ruptures` (PELT algorithm)
STL decomposition	Seasonal pattern violations — this month doesn't match expected seasonality	Time-series with known periodicity (monthly, weekly)	`statsmodels.tsa.seasonal.STL`

Linear trend break	Identifies when a KPI that was growing starts declining (or vice versa)	Time-ordered numeric data	`scipy.stats.linregress` on rolling windows
Grubbs' test	Statistically rigorous single-outlier test with p-value	Small datasets (<30 rows) where Z-score is unreliable	`scipy.stats` or manual implementation

Severity scoring logic:

- **Critical:** Outliers > 3 std deviations, concentration HHI > 0.25, changepoints in recent data, >20% missing values
- **Warning:** Outliers 2-3 std deviations, trend reversals, seasonal violations, 5-20% missing values
- **Info:** Minor distribution skew, near-duplicates, low-impact concentration, <5% missing values

When data has no anomalies: Report a "health summary" — "Your data looks clean. Here's what we checked: [list]. No significant issues found." This avoids the anti-climax of an empty screen.

Stage 2: EXPLAIN — Root Cause & Diagnostic Investigation

The goal is to answer: **"Why did this happen?"** through a guided Q&A flow. Each method produces a finding that narrows toward the root cause.

🔍 User selects a finding
'Revenue dropped 18% in Q4'

↓

Step 1: Dimension Selection

Spectra suggests top dimensions

User picks one to drill into

↓

Step 2: Contribution Analysis

Analysis

Which segments drove the change?

'73% of drop is from APAC'

Step 3: Segment Comparison

How does the problem segment differ from healthy ones?

Step 4: Correlation Scan

What other metrics moved

alongside the problem?

🧠 Root Cause Summary
'APAC revenue dropped due to Enterprise pricing change in Oct'

Method	What It Answers	How It's Used in the Q&A Flow	Python Library
Contribution analysis (additive decomposition)	"Which segments drove the overall change?" — decomposes a KPI change into per-segment contributions	Step 1: After user picks a dimension, show which segment values contributed most to the change (e.g., "APAC contributed -73% of the total decline")	`pandas.groupby` + difference math
Welch's t-test	"Is the difference between two groups statistically significant?" — compares means of two segments	Step 2: When comparing problem segment vs. rest — "APAC avg deal size ($42K) is significantly lower than other regions ($61K), p < 0.01"	`scipy.stats.ttest_ind`
Chi-squared test	"Is the distribution of categories different between	Step 2: When comparing categorical breakdowns — "Product mix in APAC	`scipy.stats.chi2_contingency`

	two groups?" — tests independence of categorical variables	is significantly different from global ($p < 0.05$)"	
Pearson / Spearman correlation	"What other metrics moved with the problem metric?" — finds co-movement	Step 3: Scan all numeric columns for correlation with the problem metric — "Customer satisfaction ($r = 0.82$) and deal close rate ($r = 0.71$) also declined"	`pandas.corr()`, `scipy.stats.spearmanr`
Decision tree (single, shallow)	"What combination of factors best predicts the problem?" — identifies the most discriminating splits	Step 3: Train a depth-2 decision tree to classify "problem rows" vs. "normal rows" — "Region=APAC AND Product=Enterprise predicts 89% of the drop"	`sklearn.tree.DecisionTreeClassifier` (max_depth=2)
Period-over-period comparison	"What changed between this period and last?" — structured diff by dimension	Step 1: When time data exists — compare current period vs. previous period across all dimensions, rank by absolute change	`pandas.groupby` + `merge`
Variance decomposition (ANOVA)	"Which dimension explains the most variance in the target metric?" — ranks dimensions by explanatory power	Pre-step: Automatically rank which dimensions to suggest first (the one that explains the most variance gets offered as the top choice)	`scipy.stats.f_oneway` or `statsmodels.stats.anova`
Pareto analysis (80/20)	"Which few segments account for most of the problem?" — identifies the vital few	Step 2: After contribution analysis — "2 of 8 regions account for 85% of the decline"	`pandas` cumulative sum math

How methods map to Q&A exchanges:

Exchange	What Spectra Asks	Statistical Method Behind It
1	"Which dimension matters most?" [options ranked by ANOVA F-statistic]	Variance decomposition ranks dimensions
2	"The decline is concentrated in [segment]. Dig deeper?"	Contribution analysis + Pareto
3	"Here's how [problem segment] differs from others."	Welch's t-test + Chi-squared
4	"These metrics also moved: [list]. Any of these relevant?"	Correlation scan
5	"Summary: [root cause statement with confidence]"	Decision tree summary + all above

---

Stage 3: MODEL — Simulation & What-If Scenarios

The goal is to answer: **"What happens if we change X?"** through a slider/input interface with projected outcomes.

🧠 Root cause identified 'APAC Enterprise pricing caused revenue drop'

↓

Step 1: Identify Levers

Spectra suggests adjustable variables from the data

↓

Step 2: Build Baseline Model

Fit relationship between levers and target KPI

levers and target KPI

Step 3: User Adjusts Levers

Sliders/inputs change variables

Model projects new outcome

Step 4: Compare Scenarios

Side-by-side: current trajectory

vs. user's scenario

↓

Step 5: Confidence Range

Show uncertainty band

not just a point estimate

Method	What It Does	When To Use	Python Library
Linear regression	Models the relationship between input variables and target KPI. User adjusts inputs → model projects new target value.	Default for all simulation — simple, interpretable, fast. "If you reduce price by 10%, projected volume increases by X based on historical relationship."	`sklearn.linear_model.LinearRegression`, `statsmodels.OLS`
Elasticity estimation	Calculates % change in outcome per % change in input (price elasticity, demand elasticity). More intuitive than raw regression coefficients.	When user adjusts price, volume, or spend levers — "Price elasticity is -1.3, meaning a 10% price cut → ~13% volume increase"	Derived from log-log regression
Time-series extrapolation	Projects the "current trajectory"	Always as the baseline	`statsmodels.tsa.holtwinters.ExponentialSmoothing`

(Holt-Winters)	baseline — what happens if nothing changes. Accounts for trend and seasonality.	comparison — "Without intervention, revenue is projected to be $X in Q2"	
Monte Carlo simulation	Generates confidence intervals by running thousands of scenarios with random variation. Shows best/worst/expected outcomes instead of a single point estimate.	After linear regression to show uncertainty — "Expected outcome: $1.1M (90% CI: $0.9M–$1.3M)"	`numpy.random` + model re-sampling
Sensitivity analysis (tornado chart)	Shows which input variable has the most impact on the outcome when changed by ±10%. Displayed as a horizontal bar chart (tornado).	Before simulation — helps user know which lever to pull first — "Price has 3x more impact than headcount on margin"	Systematic perturbation of regression inputs
Scenario comparison matrix	Side-by-side table of multiple scenarios with different input combinations. Each column is a scenario, each row is an outcome metric.	When user wants to compare 2-3 scenarios — "Conservative vs. Moderate vs. Aggressive pricing strategy"	`pandas.DataFrame` presentation
Breakeven analysis	Calculates the input value needed to hit a specific target — "What price point gets us back to $1M revenue?"	When user has a specific goal — reverse-solve the regression	Algebraic inversion of regression equation

Important design principles for Model stage:

1. **Always show confidence intervals, never just point estimates.** A single number ("revenue will be $1.1M") creates false precision. A range ("$0.9M–$1.3M with 90% confidence") is honest and builds trust.
2. **Start with linear regression, not complex ML.** Users need to understand *why* the model predicts what it does. A linear model is explainable: "For every $1 price reduction, we expect 150 more units sold." A neural network is a black box.
3. **Show the model's limitations explicitly.** "This projection assumes the historical relationship between price and volume continues. It does not account for competitor actions, market shifts, or capacity constraints." Transparency builds trust.

4. **Sensitivity analysis before simulation.** Before letting users play with sliders, show them which levers actually matter. This prevents wasted time adjusting variables that have minimal impact.

Method Availability by Data Shape

Not all methods work on all datasets. The Risk Agent must detect data shape first and only apply applicable methods.

Data Characteristic	Methods Enabled	Methods Disabled
< 30 rows	Grubbs' test, basic stats, IQR	Isolation Forest, STL, PELT, Monte Carlo (insufficient data)
No time column	All cross-sectional methods	Changepoint, STL, trend break, Holt-Winters, period-over-period
No categorical columns	Z-score, IQR, correlation, regression	Contribution analysis, Chi-squared, ANOVA, HHI
Single numeric column	Z-score, IQR, Grubbs', distribution analysis	Isolation Forest, correlation, regression, decision tree
All categorical (no numeric)	Duplicate detection, missing value patterns, Chi-squared	All numeric methods, regression, simulation
Wide data (50+ columns)	All methods, but need column selection/ranking first	Running everything on all columns (too slow, too noisy)

Library Requirements (E2B Sandbox)

These Python packages need to be available in the E2B sandbox environment:

Package	Used For	Already in Spectra?
`pandas`	Data manipulation, groupby, profiling	Yes
`numpy`	Numerical operations, Monte Carlo	Yes
`scipy`	Statistical tests (t-test, chi-squared, z-score, correlation)	Yes
`scikit-learn`	Isolation Forest, Decision Tree, Linear Regression	Yes
`statsmodels`	OLS regression, ANOVA, Holt-Winters, STL decomposition	Needs verification
`ruptures`	Changepoint detection (PELT algorithm)	Needs installation
`missingno`	Missing value pattern visualization	Optional (can use pandas)

Critical Challenges & Honest Assessment

Before committing to build, these questions need answers. Some may change the scope or sequencing.

Challenge 1: v0.8 scope is too large

The original v0.8 included: Analysis Workspace + Risk Radar + guided Q&A (basic) + Collections + PDF/MD export. That's:

- A new frontend module with its own routing and layout
- A new Risk Agent with statistical analysis logic
- A new card type (Risk Cards with severity levels)
- A guided Q&A flow (even "basic" is complex UX design)
- A Collections data model + list UI
- PDF generation pipeline

This is realistically 2 milestones of work. Cramming it into one creates pressure to cut corners on the parts that matter most (detection accuracy, Q&A flow design, report quality).

Challenge 2: The Q&A flow design has unsolved UX questions

We describe "structured Q&A with discrete choices" — but:

- **How does Spectra know what choices to offer?** It depends entirely on the data's columns and dimensions. A sales dataset with Region/Product/Channel is clean. A flat transaction log with 50 columns is not.
- **What if the data doesn't have clear categorical dimensions?** A dataset of sensor readings or financial transactions may not have natural "drill into Region" options.
- **What if there are no anomalies?** The entire flow assumes something is wrong. If the data is healthy, the Detect step shows... nothing? That's an anti-climax that needs a good UX answer.
- **How many exchanges is actually right?** We say "3-5 max" but this is a guess. Too few = shallow, too many = frustrating. This needs prototyping and user testing.

These aren't blockers — but they must be designed before building, not discovered during development.

Challenge 3: Detection accuracy makes or breaks everything

If Risk Radar has too many false positives ("flagging" normal variance as anomalies), users will stop trusting it within the first session. If it misses real issues, it's useless. The bar for proactive analysis is higher than reactive — because the user didn't ask for it, so it had better be right.

This means the Risk Agent needs careful tuning with real-world data before we ship it publicly. Synthetic test data proves the pipeline works; it doesn't prove the detection is useful.

Challenge 4: Competitive positioning is aspirational, not current

We scored Spectra at 4/4 on the capability matrix — but that's the **target after v1.0**, not reality today. Tellius has been building their Agent Mode with a large team and enterprise customers for years. We shouldn't imply we'll match Tellius feature-for-feature.

Spectra's real advantage isn't capability parity — it's accessibility:

What Tellius does better	What Spectra does better
Deeper anomaly detection (enterprise-grade ML)	Zero setup (upload a file vs. connect a data warehouse)
More sophisticated what-if modeling	Guided journey UX (not BI-platform-shaped)
Enterprise data governance and security	Deliverable-focused output (PDF/slides, not dashboards)
Real-time streaming data support	Price accessibility (individual analyst vs. $495/mo team)
Years of production hardening	Speed to first insight (minutes, not days of setup)

We win on accessibility and UX, not on depth. The positioning should reflect that honestly. "The fastest path from Excel to business insight" is a stronger message than "we do everything Tellius does."

Challenge 5: Report quality is a trust signal

If the generated PDF looks like a developer's markdown dump, users won't share it with their VP. Reports need to look polished — proper layout, branded headers, clean chart rendering, executive summary at the top. This is design work, not just engineering.

Ugly reports undermine the entire value proposition of Collections. Better to ship fewer export formats that look great than many that look mediocre.

Challenge 6: Edge cases will define whether this feels magical or broken

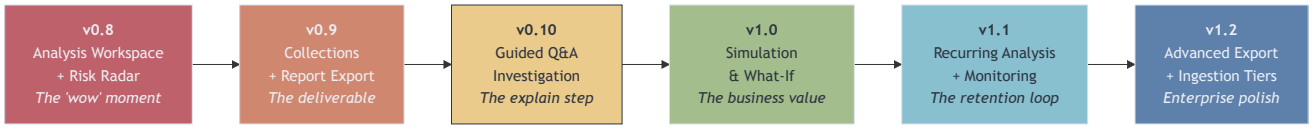
The happy path (sales data with clear dimensions, obvious anomalies, clean structure) is easy to demo. But real users will upload:

- Messy data with inconsistent formatting
- Files with only 20 rows (too few for meaningful statistics)
- Non-numeric data (text logs, categorical-only datasets)
- Data without a time dimension (no trend analysis possible)
- Multi-sheet Excel files with different structures per sheet

Each edge case needs a graceful degradation path, not a crash or misleading result.

Revised Milestone Sequence

Based on the challenges above, the original 4-milestone plan is revised into a more realistic 6-step sequence. Each milestone is smaller, more testable, and proves one thing before building the next.



Milestone	Scope	Proves	Ship Criteria
v0.8	Analysis Workspace shell + Risk Radar (Detect only). User selects data → sees findings as cards with severity and charts. No guided Q&A, no Collections yet.	The "wow" moment works. Proactive detection is accurate enough to be useful. Users understand findings without explanation.	Upload 5 different real-world datasets → detection flags useful findings in at least 4 of them with < 20% false positive rate.
v0.9	Collections module + report export (PDF + Markdown). Save any analysis (from Workspace or Chat) as a structured report. Collections list view with search/filter.	The deliverable loop works. Reports look polished enough to share with a stakeholder.	Generate 3 sample reports → a non-user rates them "would share with my boss" or higher.
v0.10	Guided Q&A investigation (the Explain step). Tap a finding → structured drill-down → root cause identification. 3-5 exchanges with discrete choices.	The guided flow is better than freeform chat for investigation. Users reach root cause faster than with Chat.	5 test scenarios with known root causes → guided flow identifies the correct driver in at least 4.
v1.0	Optimization Studio — simulation with sliders, scenario comparison, side-by-side current vs. projected.	Users trust and use the simulation. What-if feels intuitive, not intimidating.	3 test scenarios → user can set up and run a meaningful

			simulation in < 2 minutes without help.
v1.1	Recurring Analysis + Monitoring (Tier 1: manual re-upload with structure matching + automatic comparison). Email delivery of comparison reports.	Recurring value — users come back monthly. Comparison reports surface meaningful changes.	User uploads month-2 data → Spectra auto-detects match → comparison report highlights correct changes vs. month-1.
v1.2	Advanced export (PPT/slides), data ingestion Tier 2 (email forwarding) + Tier 4 (cloud storage watch), shareable report links, team views.	Enterprise readiness. Automated ingestion works reliably.	Only build what paying customers request.

What changed from the original plan

1. **Split v0.8 into v0.8 + v0.9** — Risk Radar (detection) ships first, Collections (reports) ships second. This lets us validate detection accuracy before building the report pipeline on top of it.
2. **Guided Q&A moved to v0.10** — This is the most UX-complex feature. It deserves its own milestone with dedicated design and testing, not a "basic version" squeezed into v0.8.
3. **Monitoring starts with Tier 1 only** — Manual re-upload + structure matching + comparison report. No email ingestion, no cloud watchers. Automated ingestion tiers come in v1.2 based on actual customer demand.
4. **v1.2 is demand-driven** — Advanced features only get built when paying customers ask for them. No speculative engineering.

Scope per Milestone

v0.8: Analysis Workspace + Risk Radar

Include	Exclude
Analysis Workspace as new frontend module with own nav	Guided Q&A (v0.10)
Data source picker (select from uploaded files)	Collections / report export (v0.9)
Risk Agent — auto-detection on data selection	Simulation / what-if (v1.0)
Findings as visual cards with severity (critical/warning/info)	Recurring analysis (v1.1)
Supporting charts on finding cards	PDF/Markdown export
"Healthy data" state (when no anomalies found)	Monitoring / alerts

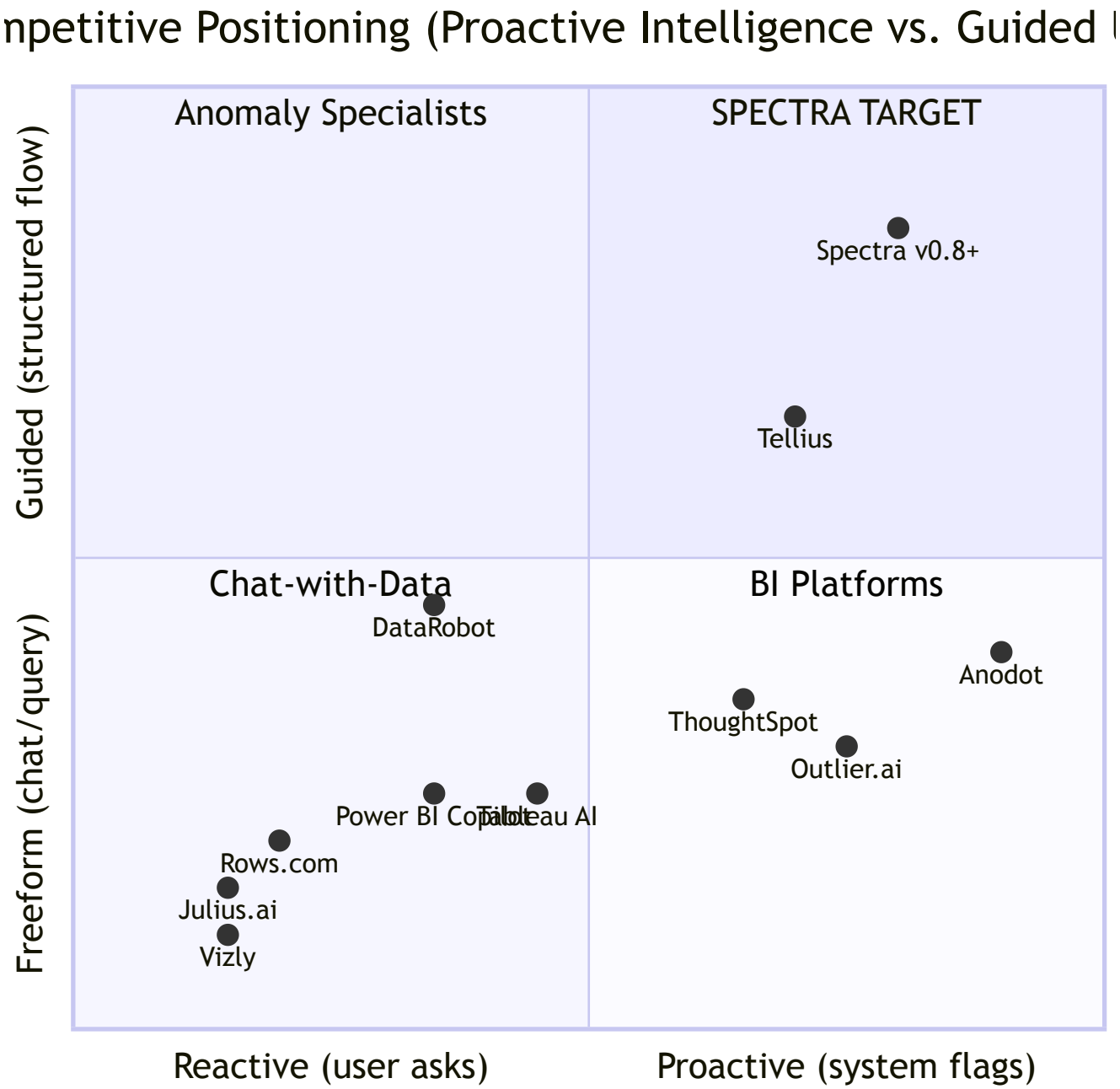
v0.9: Collections + Report Export

Include	Exclude
Collections data model (analyses, reports, metadata)	Guided Q&A (v0.10)
Save analysis from Workspace as report	PPT/slides export (v1.2)
Save analysis from Chat as report (bridge feature)	Shareable links (v1.2)
PDF export with polished layout	Team views (v1.2)
Markdown export	Recurring/monitoring (v1.1)
Collections list view with search and filter	

Competitive Landscape

The Four-Step Flow: Who Does What?

The proposed Spectra flow is: **Detect anomalies → Guided root cause investigation → Scenario simulation → Report generation.** No single product fully delivers all four today.



Head-to-Head: Full Flow Coverage

Capability	Tellius	ThoughtSpot	Anodot	DataRobot	Looker	Julius.ai	**Spectra (target)**
1. Proactive Anomaly Detection	Yes	Yes	Yes (best)	Limited	Yes	No	**Yes**

2. Guided Root Cause Investigation	Yes (Agent)	Partial	Partial	No	No	No	**Yes**
3. Scenario / What-If Simulation	Yes	No	Limited	Yes (best)	No	No	**Yes**
4. Polished Report Generation	Partial	No	No	Partial	Yes (slides)	No	**Yes**
Coverage Score	**3.5/4**	**1.5/4**	**1.5/4**	**2/4**	**1.5/4**	**0/4**	**4/4**

> ***Honest note on the 4/4 score:*** *This is the target after v1.0, not current reality. Tellius has years of production hardening with enterprise customers. Our advantage isn't matching them feature-for-feature — it's being radically more accessible. See [Honest Competitive Position](#) below.*

Category 1: AI Data Analysis (Chat-with-Data) — Direct Competitors

Product	What It Does	Proactive Detection	Guided Flow	What-If	Reports	Pricing
Julius.ai	NL data analysis + charts from uploaded files	No	No	No	Limited	Free / $20/mo Pro
Rows.com	AI-powered spreadsheet with NL analyst	Basic	No	Yes (NL)	No	Free / paid tiers
Akkio	No-code predictive model building	No	No	Limited	Limited	$49/mo+
Obviously AI	Automated ML with wizard flow	No	No	Yes	Partial	$75-145/mo
Polymer Search	Auto-analyze spreadsheets → dashboards	Basic	No	No	Limited	Free / $10/mo
Vizly	Chat-with-data + Python/R under the hood	No	No	No	Limited	Free / paid
DataChat	Conversational analytics with full reproducibility	No	Partial	No	Limited	Enterprise

Spectra's position vs. this category: These are all reactive, chat-first tools. None proactively scan data or guide users through structured investigation. The Analysis Workspace immediately differentiates Spectra from the entire category.

Category 2: Business Intelligence with AI — Enterprise Players

Product	Proactive Detection	Guided Investigation	What-If	Report Export	Pricing
ThoughtSpot	Yes (SpotIQ + anomaly alerts)	Partial (auto root cause)	No	Limited	$25/user/mo
Power BI Copilot	Limited (time-series only)	No	Limited (manual)	Yes (best: PDF/PPT/Excel)	$14/user/mo + Fabric
Tableau AI	Partial (Pulse KPI monitoring)	No	Limited (parameters)	Yes (PDF/PPT)	$42-75/user/mo

Looker (Google)	Yes (Gemini-powered)	No	No	Yes (AI-gen slides)	Enterprise custom
Sigma Computing	Yes (outlier detection + alerts)	No	No	Limited	Enterprise custom

Spectra's position vs. this category: These are massive platforms that require significant setup, data engineering, and per-seat enterprise licensing. They're adding AI features incrementally but not rethinking the workflow. Spectra can be faster to value (upload a file → instant findings) at a fraction of the cost.

Category 3: Anomaly Detection Specialists

Product	What It Does	Root Cause	Simulation	Reports	Pricing
Anodot	Autonomous ML-based business monitoring across 100% of data	Partial (pointers)	Limited (cloud cost sim)	No	Enterprise custom
Outlier.ai	Multi-dimensional anomaly scanning for executives	No	No	Limited	Enterprise custom
Sisu Data *(acquired by Snowflake, discontinued)*	Automated root cause analysis testing millions of hypotheses	Yes (best-in-class)	No	Limited	N/A

Key insight: Sisu Data was the closest to what we're building for the "Explain" step — automated root cause identification. It was acquired by Snowflake in 2023 and discontinued as standalone. That capability gap is now unfilled in the market.

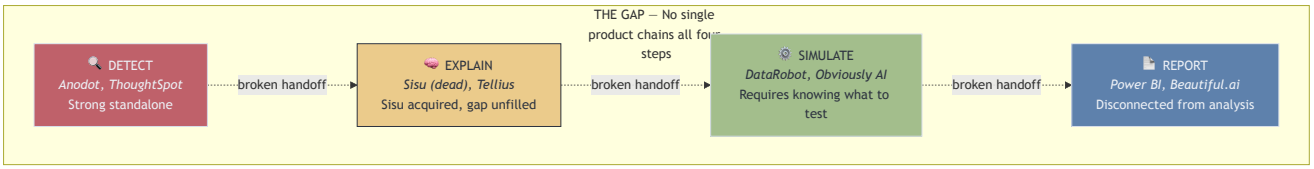
Category 4: Prescriptive Analytics / Optimization

Product	Detection	Guided Flow	What-If	Reports	Pricing
Tellius	Yes (contextual alerts)	Yes (Agent Mode Kaiya)	Yes (budget sim)	Partial (narratives)	$495/mo (5 users)
Pecan AI	Limited (data quality)	No	Limited	Limited	$950/mo+
DataRobot	Limited (model monitoring)	No	Yes (Tableau extension)	Partial	Enterprise ($100K+/yr)
H2O.ai	Yes (deep learning)	No	No	Limited	$6,900/yr+

Tellius is the closest competitor to the full Spectra vision. Their Agent Mode (Kaiya) combines anomaly detection + root cause + what-if modeling + AI narratives. However:

- At $495/mo for 5 users, it targets mid-market/enterprise
- Report output is narratives, not polished PDF/slide deliverables
- Requires data warehouse connections — not file-upload-first like Spectra
- Their UX is BI-platform-style, not guided-journey-style

The Market Gap



- **Detection specialists** (Anodot, Outlier.ai) stop at alerting — they tell you something is wrong but leave investigation to you
- **Investigation tools** (Sisu Data) have been acquired and discontinued — the capability is being absorbed into data warehouse platforms, not served standalone
- **Simulation platforms** (DataRobot, Obviously AI) require users to already know what to simulate — there's no automatic path from "here's a problem" to "here's a simulation"
- **Report tools** (Beautiful.ai, Narrative Science) are completely disconnected from the analysis — you do analysis in one tool, then manually create reports in another
- **BI platforms** (ThoughtSpot, Tableau, Power BI) are adding AI incrementally but not rethinking the end-to-end workflow

A platform that connects all four steps into a single guided flow — accessible to non-technical business users — would occupy a genuinely uncontested position.

Honest Competitive Position

Spectra's real advantage isn't capability depth — it's **accessibility and UX paradigm.** We should be honest about where we win and where we don't:

Where competitors are stronger	Where Spectra wins
Deeper anomaly detection (Anodot's enterprise ML)	Zero setup — upload a file vs. connect a data warehouse
More sophisticated modeling (DataRobot's AutoML)	Guided journey UX — not BI-platform-shaped
Enterprise governance and security (ThoughtSpot, Tableau)	Deliverable-focused output — PDF/slides, not dashboards
Real-time streaming data (Anodot, Sigma)	Price — individual analyst vs. $495/mo+ team licenses
Years of production hardening (Tellius)	Speed to first insight — minutes, not days of setup
Massive ecosystems and integrations	Simplicity — no training required, no IT involvement

Our positioning should be: *"The fastest path from Excel to business insight."*

Not: *"We do everything Tellius does."* — because we don't, and won't for a long time. We win by being the tool a business analyst can use today, alone, without asking IT for help, and produce a report their VP actually reads.

Practicality Notes

What makes this buildable now:

- Backend reuses existing E2B sandbox, agent system, and credit system — no new external services
- Risk Radar is a new agent + statistical Python code running in the same sandbox
- Collections is a new DB table + report compilation logic + PDF generation (libraries like WeasyPrint or reportlab)
- The Analysis Workspace frontend is a new Next.js route/module — separate from chat but in the same app
- Monitoring Tier 1 (manual re-upload) is structure matching + diff logic — no infrastructure

What makes this testable fast:

- Create synthetic test datasets with known anomalies and known root causes
- Upload → does Spectra find the planted issues? → does the Q&A flow reach the right answer?
- Each milestone has explicit ship criteria (defined in the milestone table above)
- Test with real-world datasets (Superstore, Kaggle sales data) not just synthetic ones

Risks to watch:

- Detection accuracy is the #1 risk — false positives kill trust faster than missing issues
- The Q&A flow needs UX prototyping before engineering — wireframe and test with users first
- Report quality (PDF layout, chart rendering) is a trust signal — ugly reports = no sharing = no value
- Simulation model accuracy (v1.0) is the hardest challenge — start with simple projections
- Edge cases (messy data, small datasets, no time dimension) need graceful degradation, not errors
- Credit cost of proactive analysis needs clear communication to users upfront